

Tema 2 — Semáforos

Fuentes: Teórica 02-Semaforos, Práctica clase-02, Guía 2, Labo 1 (+ repaso en la práctica 3). **Ejercicio 2 del parcial.**

1. Qué es un semáforo

Es un **contador de permisos** (un entero ≥ 0) más un **conjunto de threads dormidos**. Tiene dos operaciones, las dos atómicas:

- `acquire()` (también llamada `wait` o `P`): si hay permisos, toma uno y sigue. Si no hay, **se duerme** (queda *blocked*, no gasta CPU).
- `release()` (también `signal` o `V`): si hay alguien dormido, despierta a uno. Si no, suma un permiso.

En criollo: una canasta de fichas en la puerta de un boliche. Para entrar agarrás una ficha. Si no hay fichas, te sentás a esperar. El que sale devuelve la ficha, o se la da directamente a uno de los que están esperando.

Invariantes (los usás para justificar):

- $S.V \geq 0$
- $S.V = \text{inicial} + \#release - \#acquire$ (contando solo los `acquire` completados)
- Mutex con $S = \text{Semaphore}(1)$: $\#enSC + S.V = 1$. De ahí sale que $\#enSC \leq 1$ (exclusión mutua) y que no hay deadlock: si todos estuvieran bloqueados, sería $S.V = 0$ y $\#enSC = 0$, y la suma daría $0 \neq 1$.

Débiles vs fuertes (clave para los incisos de inanición)

- **Débil** (el default en el parcial y en Java: `new Semaphore(n)`): despierta a **cualquiera** de los que esperan. Encima, uno que recién llega se puede **colar** (*barging*). → Puede haber **inanición**.
- **Fuerte** (`new Semaphore(n, true)`): cola **FIFO**. Si hay N threads, a lo sumo $N-1$ te pasan adelante → **no hay inanición**.
- Mutex con semáforo débil y 3 o más threads: p y r se pueden turnar para siempre mientras q nunca entra.
- **Udding / Morris**: es posible hacer un mutex sin inanición usando solo semáforos débiles. Usa dos "puertas" y un único permiso: hasta que no se vacía la primera etapa no se arranca con la segunda.

Regla práctica: **nunca** uses `availablePermits()` para decidir algo. Entre el `if` y el `acquire` otro thread te puede ganar (check-then-act). Si necesitás "intentar sin bloquearte", existe `tryAcquire()`.

2. Los "ladrillos" (patrones mínimos)

Patrón	Código	En criollo
Señal	<code>s = Sem(0); A: hacer A; s.release(); B: s.acquire(); hacer B</code>	"avisame cuando termines"

Patrón	Código	En criollo
Rendezvous	<code>aLlego = Sem(0), bLlego = Sem(0); A: aLlego.release(); bLlego.acquire(); B: al revés</code>	"nos encontramos en la esquina y ninguno sigue hasta que llegue el otro" (labo: Andrea y Bernardo)
Mutex	<code>m = Sem(1); m.acquire(); SC; m.release()</code>	una sola llave
Multiplex	<code>s = Sem(K)</code>	"entran de a K": K fichas
Split / alternancia	<code>puedeP = Sem(1), puedeQ = Sem(0); p: puedeP.acquire(); ...; puedeQ.release()</code>	"ahora vos, ahora yo" (ping-pong)
Contador de eventos	cada A hace release, cada F hace acquire	garantiza $\#F \leq \#A$

⚠ **Pedir varios permisos del mismo semáforo de a uno → deadlock.** Si hay 2 permisos y dos threads necesitan 2 cada uno, cada uno puede tomar 1 y quedarse esperando el segundo para siempre. Solución: tomarlos bajo un mutex, o con `acquire(k)`.

3. Problemas clásicos, explicados a nivel patrón

3.1 Productor-Consumidor (buffer)

En criollo: una estantería con N lugares. Los productores ponen cosas y los consumidores sacan. No se puede sacar de una estantería vacía ni poner en una llena.

```

notEmpty = Sem(0)    // cuántas cosas hay para sacar
notFull  = Sem(N)    // cuántos lugares libres hay
mutexP = Sem(1); mutexC = Sem(1)    // un productor a la vez con el índice "inicio", un consumidor a la vez con "fin"

productor:
    notFull.acquire()           // espero lugar libre
    mutexP.acquire(); buffer[inicio] = d; inicio = (inicio+1) % N; mutexP.release()
    notEmpty.release()         // aviso que hay algo

consumidor:
    notEmpty.acquire()         // espero que haya algo
    mutexC.acquire(); d = buffer[fin]; fin = (fin+1) % N; mutexC.release()
    notFull.release()          // aviso que hay lugar

```

- `notEmpty` y `notFull` son "semáforos partidos": su suma es $\leq N$.
- Sin `mutexC`, dos consumidores pueden leer el mismo elemento y saltarse otro.
- **Ojo con el orden:** primero `notFull` y después `mutexP`. Al revés, un productor se dormiría **con el mutex tomado** y nadie más podría avanzar → deadlock.

3.2 Filósofos comensales

En criollo: 5 filósofos en una mesa redonda con 5 tenedores, uno entre cada par. Para comer necesitás los dos tenedores de al lado. Si todos agarran el izquierdo al mismo tiempo, todos esperan el derecho

para siempre: **deadlock** (espera circular).

Soluciones:

- **(A) Reducir la concurrencia:** `sillas = Sem(N-1)` . Como a lo sumo 4 se sientan, siempre queda un tenedor libre y el ciclo se rompe. Sin inanición si `sillas` es fuerte.
- **(B) Romper la simetría:** un filósofo toma primero el derecho. Generalización: **tomar los recursos siempre en un orden global** (el de menor número primero).
- (Con monitores, tema 3: se toman los 2 tenedores juntos, atómicamente.)

Lección general: hay deadlock cuando hay **espera circular**. Se evita con orden total de adquisición, con límite de participantes, o tomando todo junto.

3.3 Lectores–Escritores ★ (el más importante)

En criollo: una biblioteca. Pueden leer muchos a la vez, pero el que escribe tiene que estar solo, sin lectores ni otros escritores.

a) Prioridad lectores = patrón LIGHTSWITCH (interruptor de luz):

```
cantLectores = 0; mutexL = Sem(1); escribir = Sem(1)    // escribir = "room vacío"

lector:
    mutexL.acquire()
    cantLectores++
    if (cantLectores == 1) escribir.acquire()    // el primero prende la luz
    mutexL.release()
    LEER
    mutexL.acquire()
    cantLectores--
    if (cantLectores == 0) escribir.release()    // el último la apaga
    mutexL.release()

escritor:
    escribir.acquire(); ESCRIBIR; escribir.release()
```

Lightswitch: el primero que entra a la pieza prende la luz (toma `escribir`), el último que sale la apaga (lo libera). Mientras haya lectores, la luz queda prendida y el escritor no puede entrar.

Problema: si siempre llega algún lector antes de que se vacíe la pieza, **el escritor espera para siempre** (inanición).

b) Sin inanición = agregar un MOLINETE (turnstile):

```

molinete = Sem(1, true) // FUERTE
lector:
    molinete.acquire(); molinete.release() // pasar por el molinete
    ...lo mismo de antes...

escritor:
    molinete.acquire() // me paro en el molinete
    escribir.acquire()
    ESCRIBIR
    molinete.release()
    escribir.release()

```

Turnstile: el molinete de la entrada al subte. Todos pasan de a uno (acquire + release). Cuando el escritor llega, **se queda parado en el molinete** (lo toma y no lo suelta), así que los lectores nuevos no pueden pasar. Los que ya estaban adentro terminan y el escritor entra. Tiene que ser **fuerte** para que la garantía sea formal.

c) Prioridad escritores: si hay un escritor esperando, los lectores nuevos no entran.

- Los escritores usan su propio lightswitch sobre un semáforo `leer`: el primer escritor que llega lo toma y el último lo suelta.
- Los lectores tienen que pasar por `leer` para entrar.
- `mutexP` hace que haya a lo sumo un lector esperando en `leer`, para que el escritor gane siempre.

3.4 Puente de una vía / Saldungaray (labo 1, Guía 2 ej 12)

En criollo: es lectores-escritores **simétrico**. Autos del mismo sentido pueden ir juntos; del sentido contrario, no.

- **Un lightswitch por sentido**, los dos sobre el mismo `resource`.
- **Turnstile compartido:** el primer auto de B que espera en `resource.acquire()` retiene el molinete, así el sentido A deja de recibir autos y se vacía.
- Tiene que ser **fuerte** para que ningún auto espere para siempre. El apunte avisa: "*Tenerlo presente si piden justificar no starvation con precisión (Ejem, un parcial, ejem)*".
- **"Máximo K en el puente":** + multiplex `Sem(K)`.
- **"No se pasan entre sí" (FIFO):** paso de testigo (3.8).

3.5 Barrera (Volcán Lanín, labo 1)

En criollo: nadie arranca el siguiente tramo hasta que llegan todos a la pirca.

```

mutex.acquire(); count++; if (count == n) turnstile.release(n); mutex.release();
turnstile.acquire()
mutex.acquire(); count--; if (count == 0) turnstile2.release(n); mutex.release();
turnstile2.acquire()

```

El segundo molinete está para que un porteño "ansioso" no arranque la ronda siguiente y cuente mal mientras los otros todavía están saliendo de la anterior.

3.6 Fumadores (Patil)

En criollo: para armar un cigarrillo hacen falta tabaco, papel y fósforos. Cada fumador tiene infinito de **una** cosa. El agente pone **dos** cosas sobre la mesa. Tiene que agarrarlas el fumador al que le faltan justo esas dos.

- La solución ingenua (cada fumador hace `acquire` de las dos cosas que le faltan) **da deadlock**: un fumador agarra el papel, otro agarra los fósforos, y ninguno completa su par.
- **Solución: threads "gestores"**. Uno por ingrediente, con booleanos `hayX` bajo un mutex. Cuando llega un ingrediente, el gestor se fija si ya estaba el otro. Si estaba, despierta al fumador correcto; si no, marca el suyo como disponible.

Lección: cuando la condición es **compuesta** ("A y B a la vez"), los `acquire` sueltos no alcanzan. Hace falta una variable de estado bajo un mutex (o un monitor, tema 3: el "Mostrador").

3.7 Barbero dormilón (Dijkstra)

En criollo: una peluquería con un barbero y n sillas de espera. Si no hay nadie, el barbero duerme. El cliente que llega y encuentra todo lleno se va.

- Contador `genteEnBarberia` bajo un mutex: **"si está lleno, me voy"** se decide con el contador, no con `availablePermits`.
- Dos pares de semáforos de señal: `solicitudCorte` / `listoParaCortar` para arrancar el corte, y `corteTerminado` / `clienteRetirado` para cerrarlo.
- Para atender **en orden de llegada** sin semáforos fuertes: cada cliente pone *su propio* semáforo en una cola y el barbero los va liberando de a uno (3.8).

3.8 Paso de testigo (baton passing) = FIFO sin semáforos fuertes

En criollo: en vez de que todos esperen en la misma puerta (donde el portero deja pasar a cualquiera), cada uno tiene **su propio timbre**. Cuando te toca, te lo tocan a vos.

```
anotarse(): mutex.acquire(); s = Sem(0); primero = fila.vacia(); fila.addLast(s); mutex.release()
            if (primero) s.release(); return s      // el thread después hace s.acquire()
avisarAlSiguiente(): mutex.acquire(); fila.removeFirst(); sig = fila.peekFirst(); mutex.release()
                    if (sig != null) sig.release()
```

Invariante: entre los que están en la fila, a lo sumo un semáforo está abierto, y es el del más antiguo. Este patrón generalizado es un **monitor hecho a mano** (tema 3).

3.9 Venta de tickets (labo 1)

- Lectores-escritores (muchos miran el stock, uno compra) + un **número de versión**.
- Compro solo si la versión que leí sigue siendo la actual; si no, releo y reintento. Es la idea "optimista" que vuelve en el tema 4.

4. Checklist para justificar una solución con semáforos

1. Para qué es cada semáforo, con qué valor arranca, y **qué invariante** mantiene.
2. **Sin race conditions:** cada variable compartida se toca con su mutex tomado.
3. **Sin deadlock:**
 - no hay espera circular (orden global);
 - nadie se duerme con un mutex tomado que necesita quien lo va a despertar;
 - todo acquire tiene su release en todos los caminos.
4. **Inanición:** con semáforos débiles puede haber sobrepasos sin cota (mostrá la traza). Con fuertes, la cota es la cantidad de threads que ya estaban adelante en la cola.
5. Suposiciones explícitas: "asumo semáforos débiles", "el turnstile es fuerte porque...".

5. Java (práctica 2 / labo)

- **Crear threads:** `new Thread(() -> ...).start()`.
 - `.run()` **no** crea un thread nuevo, ejecuta en el mismo.
 - `join()` espera a que termine. Primero lanzá todos y después hacé todos los joins.
- `interrupt()`: pide parar, de forma cooperativa. No te comas la `InterruptedException`: relanzala o volvé a prender el flag.
- `volatile`: visibilidad, no atomicidad.
- **ReentrantLock**:
 - `lock()` y `unlock()` **en un finally**.
 - Tiene **dueño**: solo lo libera el que lo tomó.
 - Es reentrante (el mismo thread lo puede volver a tomar).
 - `new ReentrantLock(true)` es fair.
- **Semaphore**: no tiene dueño (cualquiera puede hacer release) y no es reentrante. Métodos: `acquire(k)`, `tryAcquire()`, `new Semaphore(n, true)` para el fuerte.
- **Por dentro** (cultura general): AQS (un entero atómico con CAS + una cola) → `LockSupport.park()` → futex del sistema operativo.

6. Guía 2 — qué patrón entrena cada ejercicio

Ej	Patrón
1–3	señales / precedencia (ya los resolviste)
4	contadores de eventos ($\#F \leq \#A$)
5	split semaphores (alternancia, diferencia ≤ 1 , ABB)
6	señalización generador ↔ acumulador (en Java)
7	gimnasio: recursos múltiples (aparato + k discos): orden de adquisición / tomar k juntos
8	bolitas: capacidad que cambia con los participantes + consumidores que toman pares (¡deadlock!)
9	bote: barrera / "esperar N a bordo" + dos costas
10	planta: señales entre vehículo y máquina (rendezvous)
11	baño + limpieza: lightswitch (a), turnstile / prioridad (b), dónde va el lightswitch (c) ★
12	punto: lightswitch por sentido (a), + multiplex (b), inanición (c) ★